Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R Narasimhan. 2024. SWE-bench: Can language models resolve real-world github issues? In *The Twelfth International Conference on Learning Representations*.

Haoyang Li, Xuejia Chen, Zhanchao XU, Darian Li, Nicole Hu, Fei Teng, Yiming Li, Luyu Qiu, Chen Jason Zhang, Qing Li, and Lei Chen. 2025. Exposing numeracy gaps: A benchmark to evaluate fundamental numerical abilities in large language models. *Preprint*, arXiv:2502.11075.

Jack Lindsey, Wes Gurnee, Emmanuel Ameisen, Brian Chen, Adam Pearce, Nicholas L. Turner, Craig Citro, David Abrahams, Shan Carter, Basil Hosmer, Jonathan Marcus, Michael Sklar, Adly Templeton, Trenton Bricken, Callum McDougall, Hoagy Cunningham, Thomas Henighan, Adam Jermyn, Andy Jones, and 8 others. 2025. On the biology of a large language model. *Transformer Circuits Thread*.

Kyle Lo, Lucy Lu Wang, Mark Neumann, Rodney Kinney, and Daniel Weld. 2020. S2ORC: The semantic scholar open research corpus. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 4969–4983, Online. Association for Computational Linguistics.

Ilya Loshchilov and Frank Hutter. 2019. Decoupled weight decay regularization. *Preprint*, arXiv:1711.05101.

Luca Soldaini and Kyle Lo. 2023. peS2o (Pretraining Efficiently on S2ORC) Dataset. Technical report, Allen Institute for AI. ODC-By, https://github.com/allenai/pes2o.

Xingyou Song and Dara Bahri. 2025. Decoding-based regression. *Computing Research Repository*, arXiv:2501.19383.

Alessandro Stolfo, Yonatan Belinkov, and Mrinmaya Sachan. 2023. A mechanistic interpretation of arithmetic reasoning in language models using causal mediation analysis. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*.

Eric Tang, Bangding Yang, and Xingyou Song. 2025. Understanding LLM embeddings for regression. *Transactions on Machine Learning Research*.

Ross Taylor, Marcin Kardas, Guillem Cucurull, Thomas Scialom, Anthony Hartshorn, Elvis Saravia, Andrew Poulton, Viktor Kerkez, and Robert Stojnic. 2022. Galactica: A large language model for science. *Preprint*, arXiv:2211.09085.

Eric Wallace, Yizhong Wang, Sujian Li, Sameer Singh, and Matt Gardner. 2019. Do NLP models know numbers? Probing numeracy in embeddings. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 5307–5315, Hong Kong, China. Association for Computational Linguistics.

Shijie Wu, Ozan Irsoy, Steven Lu, Vadim Dabravolski, Mark Dredze, Sebastian Gehrmann, Prabhanjan Kambadur, David Rosenberg, and Gideon Mann. 2023. BloombergGPT: A large language model for finance. *Preprint*, arXiv:2303.17564.

Haotong Yang, Yi Hu, Shijia Kang, Zhouchen Lin, and Muhan Zhang. 2025. Number cookbook: Number understanding of language models and how to improve it. In *The Thirteenth International Conference on Learning Representations*.

Qiang Zhang, Keyan Ding, Tianwen Lv, Xinda Wang, Qingyu Yin, Yiwen Zhang, Jing Yu, Yuhao Wang, Xiaotong Li, Zhuoyi Xiang, Xiang Zhuang, Zeyuan Wang, Ming Qin, Mengyao Zhang, Jinlu Zhang, Jiyu Cui, Renjun Xu, Hongyang Chen, Xiaohui Fan, and 2 others. 2025. Scientific large language models: A survey on biological & chemical domains. *ACM Comput. Surv.*, 57(6).

Fangwei Zhu, Damai Dai, and Zhifang Sui. 2025. Language models encode the value of numbers linearly. In *Proceedings of the 31st International Conference on Computational Linguistics*, pages 693–709, Abu Dhabi, UAE.

A Related Work

There is rich recent literature on internal numerical representations in large language models. Wallace et al. (2019) demonstrate that numerical information can be extracted from contextual or static word embeddings. Stolfo et al. (2023) and Hanna et al. (2023) study internal mechanisms of LLMs' mathematical abilities. Closely related to our work is Zhu et al. (2025), who provide evidence that language models encode the value of integers linearly in log-space. Most recently, Lindsey et al. (2025) attempted to recover the internal procedure through which a specific LLM performs additions of two-digit numbers.

Our work also overlaps with recent LLM numeracy benchmark efforts. Recently, Yang et al. (2025) designed a comprehensive benchmark for evaluating LLMs' black-box responses to basic synthetic numerical questions. However, they did not investigate internal numerical representations. Similarly, Li et al. (2025) designed a basic numerical benchmark based on real-world datasets, using a similar blackbox approach. In a similar effort to extract precise numbers from LLM encodings, Tang et al. (2025) and Song and Bahri (2025) studied whether LLM encodings can be used to tackle classical regression tasks.

B Data Setup

B.1 Synthetic Data

The `cross-notation` dataset in §2 contains 11,200 comparison problems, with 8,000 samples for training, 1,600 for validation and 1,600 for evaluation. We generate 1,400 numeral pairs for each digit length between 2 and 9 digits, randomly converting one numeral in each pair to scientific notation while keeping the other in standard notation.

We construct two variants of this dataset: `int-sci` (integer vs. scientific notation) and `dec-sci` (decimal vs. scientific notation). In `int-sci`, both numbers start as integers before one is converted to scientific notation (e.g., 570 vs. $580 \rightarrow 5.7 \times 10^2$ vs. 580). In `dec-sci`, we first append 0–4 random decimal digits independently to each number before conversion (e.g., 570.23 vs. $871.6 \rightarrow 570.23$ vs. 8.716×10^2). When zero decimal digits are selected, we append ".0" to maintain consistent formatting (e.g., 342.0). These two datasets are functionally interchangeable for our experiments. We report results using `dec-sci` in Figs. 3, 10, 11, 13 and 14, with `int-sci` used for all other experiments on `cross-notation`.

B.2 Scientific Data

We utilize the `peS2o` dataset (Soldaini and Lo, 2023), which contains approximately 40 million open-access academic papers from arXiv that have been cleaned, filtered, and formatted for language model pre-training. `peS2o` consists of two subsets: `s2orc`, which provides full-text papers (Lo et al., 2020), and `s2ag`, which contains titles and abstracts only.

To build `arXiv` data, we only use the `s2orc` subset. We first shuffle the corpus using `terashuf` and select the first 100k articles from `s2orc`. We then employ regular expressions to extract numerals in standard decimal notation (`(?<!\d\.)\d+\.\d+(?!\.\d)`) and in scientific notation (`(?:\d+(?:\.\d+)?)\s*×\s*10\s+[-+]?\d+`). To avoid overflowing GPU memory, we filter out papers exceeding 30,000 tokens. We process complete documents through the language models and extract hidden states at the final token position of each identified numeral. The resulting processed `arXiv` dataset contains 5,000 samples, divided into 4,000 training samples and 500 validation samples and 500 evaluation samples.

C Inference and Prompting Setup

For verbalization tasks, all models are evaluated with temperature 0 (greedy decoding) to ensure deterministic and reproducible outputs.

C.1 Open-weight Models

For the 7B–8B open-weight models, we use the Hugging Face `Transformers` library for decoding. Prompts are formatted as plain text with "Q:" and "A:" markers. The one-shot demonstration for `int-sci` is shown in §2. For `dec-sci`, we use:

```
Q: Which is larger, 9.9 × 10^2 or 899.9? A: 9.9 × 10^2
Q: Which is larger, {a} or {b}? A:
```

For k-shot experiments using `int-sci` (§H), where $k \in [1, 5]$, we prepend the first k of these examples:

```
Q: Which is larger, 9.9 × 10^2 or 100? A: 9.9 × 10^2
Q: Which is larger, 161230 or 7.182 × 10^5? A: 7.182 × 10^5
Q: Which is larger, 713 or 4.78 × 10^2? A: 713
Q: Which is larger, 1.354 × 10^6 or 4906723? A: 4906723
Q: Which is larger, 20834 or 6.5 × 10^3? A: 20834
```

followed by the usual test question:

```
Q: Which is larger, {a} or {b}? A:
```

C.2 GPT Models

GPT-4.1 and GPT-4.1-mini are called via the OpenAI Chat Completions API with the system prompt "Just answer with a number." The questions and answers are provided as messages with the `user` and `assistant` roles respectively, rather than being marked with `Q:` and `A:` strings. The content of the few-shot examples is identical to that above.

D Probing Setup

D.1 Obtaining Hidden States

We feed each input into the LLMs and extract the hidden states at every layer. For each layer, we identify specific token positions corresponding to our target representations and save their hidden states $\boldsymbol{H} \in \mathbb{R}^{n \times d_{\text{model}}}$ for subsequent probing experiments, where n is the number of samples and d_{model} is the hidden dimension.

D.2 Training Regression Probes

For regression probes, we extract hidden states from the last token of each numeral. Given hidden states $\boldsymbol{H}$ and their corresponding numerical values $\boldsymbol{x} \in \mathbb{R}^n$, we train a linear regressor $\mathcal{R}$ that predicts $\boldsymbol{y} = \boldsymbol{H}\boldsymbol{w} + b\,\mathbf{1}$, where $\boldsymbol{w} \in \mathbb{R}^{d_{\text{model}}}$ and $b \in \mathbb{R}$ are the learned parameters.

We apply $\log_2$ transformation to compress the numerical range and emphasize relative magnitude relationships, using an ℓ_2 regularizer ($\lambda = 1$) for training (i.e., ridge regression):

$$\boldsymbol{w}^*, b^* = \underset{\boldsymbol{w}, b}{\arg\min} \, || \log_2(\boldsymbol{x}) - \boldsymbol{H}\boldsymbol{w} - b\,\mathbf{1}||_2^2 + \lambda ||\boldsymbol{w}||_2^2$$

The trained probe predicts logarithmic magnitudes as $\boldsymbol{y} = \boldsymbol{H}\boldsymbol{w}^* + b^*\,\mathbf{1}$.

D.3 Training Classification Probes

For classification probes, we extract hidden states from the last token of the entire prompt after processing both numerals. We frame the task as binary classification to predict which numeral is larger in a given pair (a, b).

Given hidden states $\boldsymbol{H} = \{\boldsymbol{h}_1, \ldots, \boldsymbol{h}_n\} \in \mathbb{R}^{n \times d_{\text{model}}}$ and binary labels $\mathbf{y} \in \{0, 1\}^n$ (where $y_i = 1$ if the first numeral is larger), we train a logistic regression model with an ℓ_2 regularizer ($\gamma = 1$):

$$\boldsymbol{w}^*, b^* = \underset{\boldsymbol{w}, b}{\arg\max} \left\{ \sum_{i=1}^{n} y_i \log(\sigma(\boldsymbol{h}_i^\top \boldsymbol{w} + b)) + (1 - y_i) \log(1 - \sigma(\boldsymbol{h}_i^\top \boldsymbol{w} + b)) - \frac{1}{2\gamma} \|\boldsymbol{w}\|^2 \right\}$$

where σ is the sigmoid function. Given a hidden state $\boldsymbol{h} \in \mathbb{R}^{d_{\text{model}}}$, the probe estimates $P(a > b) = \sigma(\boldsymbol{h}^\top \boldsymbol{w}^* + b^*)$. Our binary classifier predicts $a > b$ iff this probability exceeds $\frac{1}{2}$, i.e., iff $\boldsymbol{h}^\top \boldsymbol{w}^* + b^* > 0$.